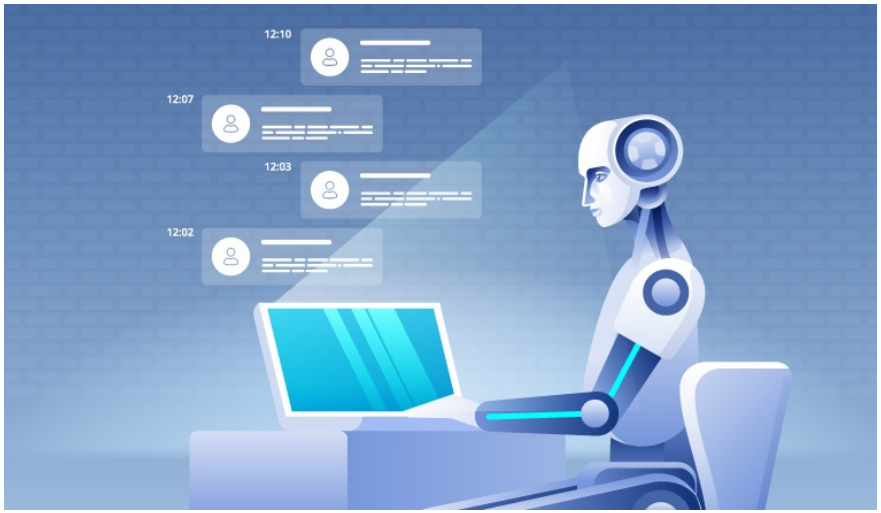
Agenda:

1. Review and Questions
2. Natural language Processing
3. Word embedding
4. Practical example on NLP

NLP

Natural language processing

NLP Applications:



Applications:

Speech Recognitions

Machine translation

Context, sentiment and topic analysis/classification

- *Spam and junk email filtering (e.g. gmail)*
- *Product and services review processing and classification, (e.g. amazon)*
- *Product and services recommenders, (e.g. IMDB)*

Answering questions (Chatbot)

Translation (e.g. google translate)

Grammar correction (e.g. Microsoft Word)

Next word suggestion (e.g. your massaging app in your smart phone)

Voice/speech recognition and voice response systems (e.g personal assistant like Siri, Alexa…)

More…

Natural language processing (NLP):

As an example these are the steps a machine can interact with humans

1. *Machine record human voice*
2. *Machine turn the recorded signal (Audio) into text*
3. *Machine analyse the text using NLP*
4. *Machine will respond*

….

Lets look at few applications of NLP:

https://www.youtube.com/watch?v=LY7x2Ihqjmc
https://www.youtube.com/watch?v=vUgUeFu2Dcw

Music: https://soundcloud.com/tigerandman/home-on-the-land

How NLP works:

Challenges:

Word to word translation (early models):

- This sentence from bible “the spirit is willing, but the flesh is weak” in translation in Russian “the vodka is agreeable, but the meat is spoiled”
- “out of sight, out of mind” to Chinese -> “Invisible idiot” or Russian ”blind and insane”

NLP resolved lots of these problems, but there are still other challenges:

- Grammar rules,
- Real context, slang, words with different meanings, plural forms…
- Structured data vs unstructured data,
- Properly breaking it down: Words (proper vocabulary) vs sentences vs paragraphs (different meanings!) and part of speech tags (POS)
- Named entities, like names, locations, numbers…
- Emotions,
- Different accents,
- …

How NLP works:

How translate natural language to machine language?

We need to represent our language in a way that machine can make sense of it.

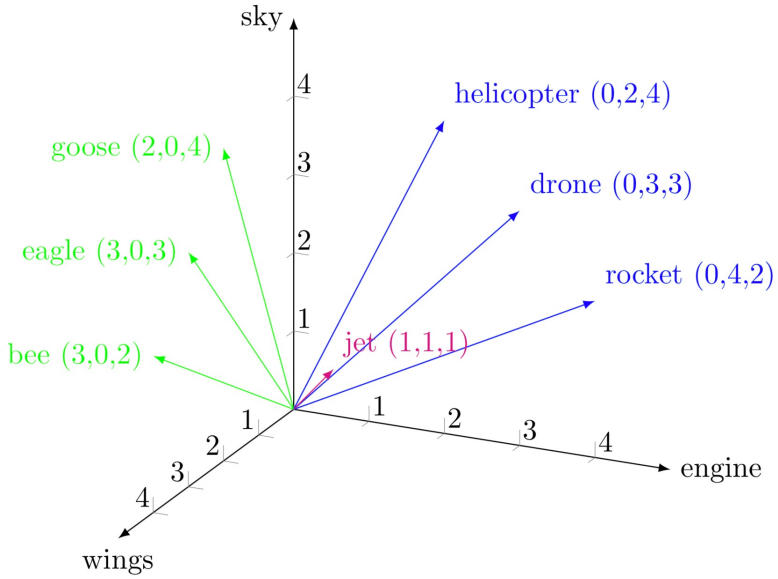
How machines computes make sense of things?

Challenges?

Tools?

Word embedding:

Word Embedding is a technique used in NLP to represent words in machine language for text analysis. Words are encoded in real-valued vectors such that words sharing similar meaning and context are clustered together and are closer to each other in vector space.



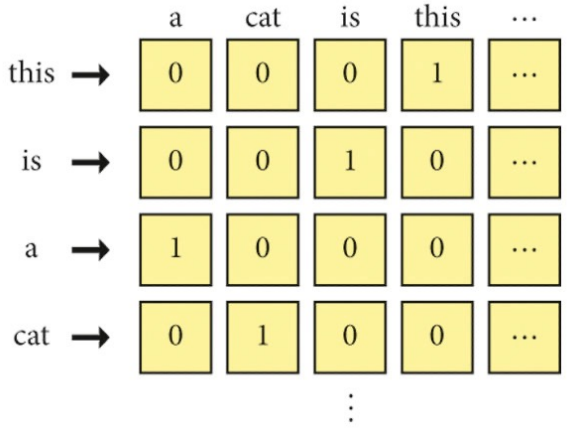
How NLP works:

There are tools and techniques, today we will learn some of them:

- One hot encoder
- Unique numbers
- Bag of word
- TF-IDF
- N-Gram
- Word2Vec
- GloVe
- …

Data gathering and pre-processing: "**one hot encoding**"

- In NLP, One hot encoding is a vector representation of words in a vocabulary.
- Each word in the vocabulary is represented by a vector of size 'n', where 'n' is the total number of words.

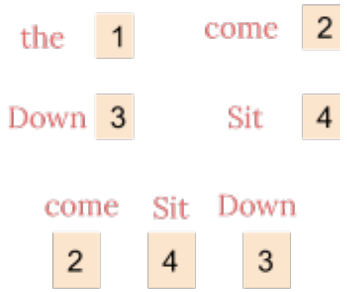


Data gathering and pre-processing: "**integer encoding**"

- Every word is represented by a unique number,

- Problem with this?

  Random and no relationship between words



Data gathering and pre-processing: "**Bag of Words**"

- Bag of words, and tokenizing the words (features)
- Simply: Break your sentences into words (or characters, punctuations…), THAT'S IT!
- Note that what you refer to as word is different based on the application, examples from our area:
  - SLAR, MODAR, PT, CTS… all of these can be words
  - Numbers and letters: B3L08, B8S19
  - More?

Bag of words Example: consider the following sentences:

1. "Nuclear energy can save our planet!",
2. "Nuclear energy is a clean energy!!",

	a	can	clean	energy	is	nuclear	our	planet	save	1	2	.	!	a	b	…
Sentence 1	0	1	0	1	0	1	1	1	1	1	0	1	1	4	0	…
Sentence 2	1	0	1	2	1	1	0	0	0	0	1	1	2	3	0	…

- It can be noted that bag of words can be in words, numbers, characters, punctuations…

Data gathering and pre-processing: "**Bag of Words**"

- Think about an email spam filter, you may want to look for the word "spam", what if the hacker use these words for example:

  "spAm", vs "s p a m" vs "sppam" vs "spåm" vs" "ʃpam"

- How do you want to make sure you capture all of these?
- Problem: Bag of Words weight all words the same way.
  - In our previous example word "nuclear" appears in both example so it is a key word!
  - How do you differentiate that?

Data gathering and pre-processing: "**TF-IDF**"

- TF-IDF, Term Frequency-Inverse Document Frequency

$$\text{Term frequency} = \frac{\text{Frequency of } \textbf{the word} \text{ in a sentence or sample}}{\text{total No of words in the sentence}}$$

$$\text{IDF} = \log\left(\frac{\text{total No of sentence or sample}}{\text{No of sentences or samples that contain } \textbf{the word}}\right)$$

- So, for the world "nuclear" we have

$$TF_{sentence\ 1} = \frac{1}{6},\ TF_{sentence\ 2} = \frac{1}{5}$$

$$IDF = \log\left(\frac{2}{2}\right)$$

- Using TF-IDF different words like "energy", "Nuclear" and "clean" are differentiated.
- Note that same can be applied to Characters, punctuations…
- **Problem**: With Bag of Words and TF-IDF, both context and the sequence information is lost, example:

1) "it is sunny out there, there is no rain"
2) "it is rainy out there, there is no sun"

Using Bag of words and TF-IDF, both of these sentences have the same matrix representation, although they are have different meanings.

Data gathering and pre-processing: "**N-Gram**"

- To solve the problem and not lose to much context information we can use N-Grams model.
  - N-Grams: Simply is the continues sequence of N items, (N is an integer)
- Looking back at our example sentences:

1) "High power at low EFPH"
2) "Low power at high EFPH"

With N=2, new features (2-Grams) are:

1) "Hight power", "Power at", "at low", "low EFPH"
2) "Low power", "Power at", "at high", "high EFPH"

- It can be seen that the two sentences don't have the same matrix representation any more.
- With N=3, features are: "High power at", "power at low", "at low EFPH"

Challenges:

Techniques such as bag of words, TF-IDF, unique numbers… have their challenges, what are they?

Problems with bag of words:

- Vocabulary: How to design the vocabulary, what to include and what not
- Sparsity: For big models (large data representation) the matrix can be very sparse. Challenges with computation, space, time
- Meaning & relationships: relationship between words can be lost. Different context (sentences) may have same vector representations, example?

Word2Vec:

- Word2vec is a method to efficiently create word embeddings by using a two-layer neural network.
- Developed by Tomas Mikolov, et al. at Google in 2013 in order to make the neural-network-based training of the embedding more efficient.
- The input of word2vec: text corpus
- The output of word2vec: is a set of vectors known as feature vectors that represent words in that corpus.

Word2vec is a shallow neural network, that convert text into a numerical form that deep neural networks can understand.

Word2Vec addresses that challenges that exist in bag of words etc…

- Instead of having a feature vector for each input with a length equal to the size of the vocabulary, each token becomes a vector with a length
- Instead of vectorizing a token itself, Word2Vec vectorizes the context of the token by considering neighboring tokens

Word2Vec:

Word2vec is uses 2 techniques to learn the word embedding:

1. Continuous Bag-of-Words (CBOW model)
2. Skip-Gram model

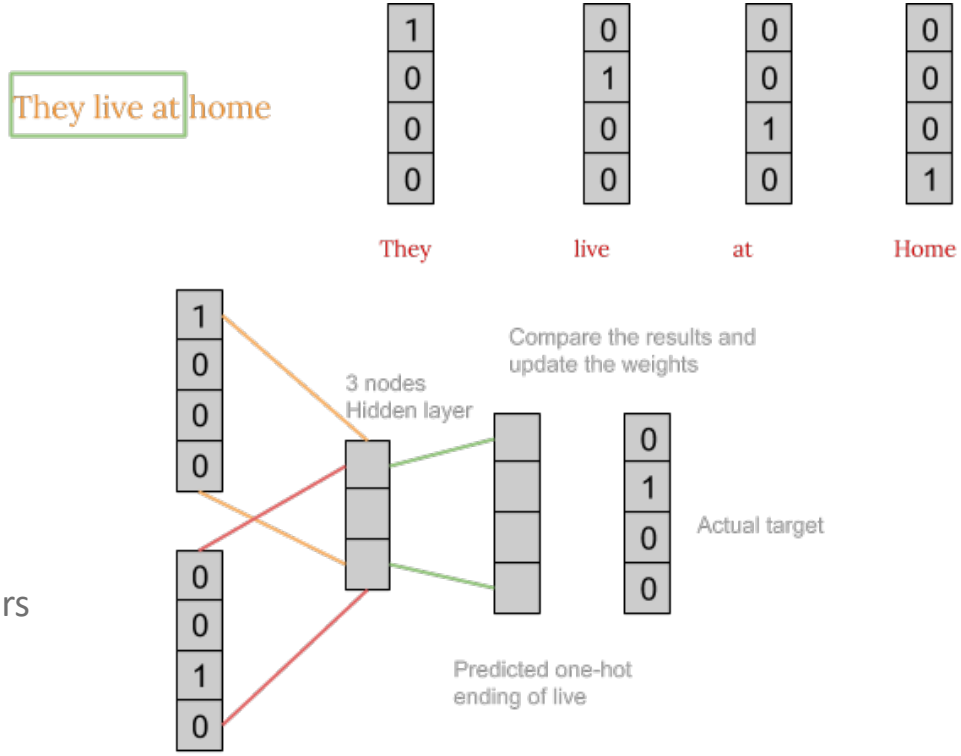
Word2Vec:

Continuous Bag-of-Words (CBOW model):

CBOW learn the embedding and predict the probability of a word to occur based on the context (surroundings words).

In this example we predict the probability of the centre word “Live” to occur based on “They” and “at”

(Data is created using the sliding window, pairs of (centre, surroundings) are extracted and words are converted using one hot encoder)



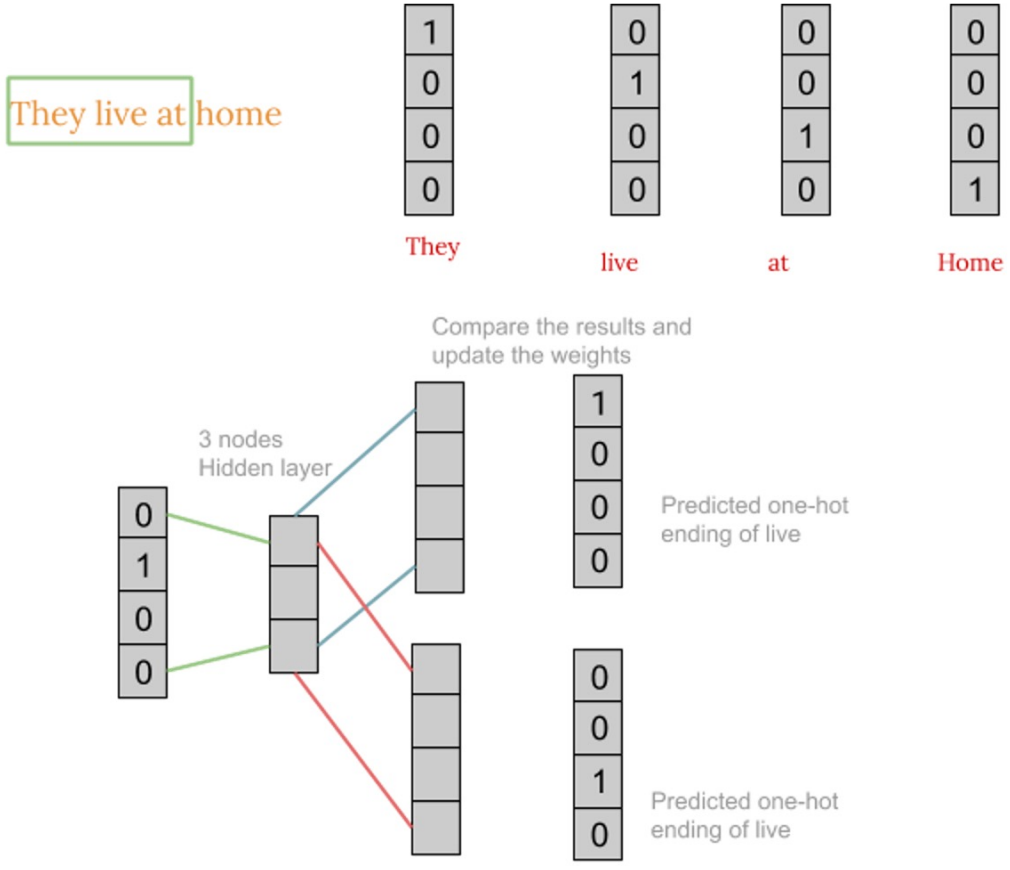
Word2Vec:

Skip-Gram model

This model works in a reverse fashion as CBOW.

Skip-Gram learn to predict the surroundings words based on the target word.

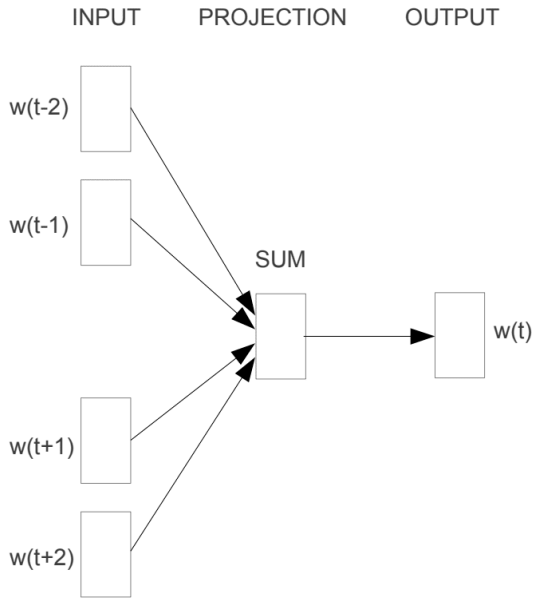
In this example we predict the probability of the words “They” and “at” based on “Live”



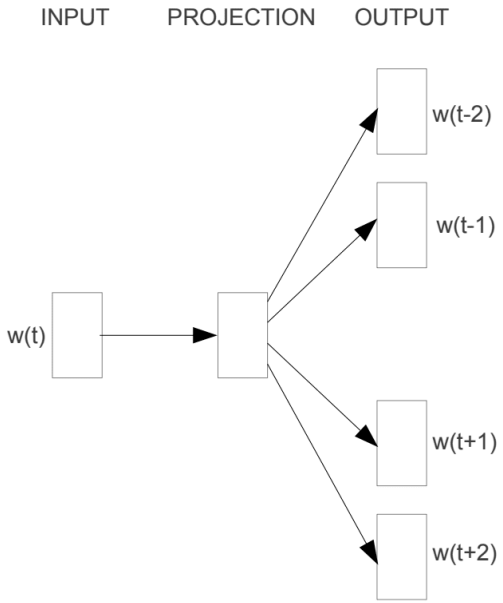
Word2Vec:

Word2vec is uses 2 techniques to learn the word embedding:

1. Continuous Bag-of-Words (CBOW model)
2. Skip-Gram model



CBOW



Skip-gram

Word2Vec:

Word2vec is based on the sliding window and uses ”local information” (i.e. local statistics)
Is this a problem?

Example: “The cat sat on the mat”

Word2vec, would not be able to easily answer:

- is “the” a special context of the words “cat” and “mat”?

or

- is “the” just a stop word?

GloVe:

GloVe (Global Vectors) is an unsupervised learning algorithm for obtaining vector representations for words.

- GloVe captures both global statistics and local statistics of a corpus, in order to come up with word vectors. It combines the advantages of the two major model families in the literature:
  - global matrix factorization and,
  - local context window methods

- GloVe constructs an explicit word-context or word co-occurrence matrix using statistics across the whole text corpus (Rather than using a sliding window to define local context).

GloVe:

Lets construct the word co-occurrence matrix for this example:

- *"I like deep learning"*
- *"I like NLP"*
- *"I enjoy flying"*

The model efficiently leverages statistical information by training only on the nonzero elements in a word-word co- occurrence matrix, rather than on the entire sparse matrix or on individual context windows in a large corpus.

counts	I	like	enjoy	deep	learning	NLP	flying	.
I	0	2	1	0	0	0	0	0
like	2	0	0	1	0	1	0	0
enjoy	1	0	0	0	0	0	1	0
deep	0	1	0	0	1	0	0	0
learning	0	0	0	1	0	0	0	1
NLP	0	1	0	0	0	0	0	1
flying	0	0	1	0	0	0	0	1
.	0	0	0	0	1	1	1	0

Python:

Lets make our first NLP:

A SMS spam detector

UCI directory: http://archive.ics.uci.edu/ml/datasets/SMS+Spam+Collection#

Sample data:

Ham (non-spam): What you doing?how are you?

Spam: FreeMsg: Txt: CALL to No: 86888 & claim your reward of 3 hours talk time

Python:

Patient’s Drug Review Dataset

UCI directory:
https://archive.ics.uci.edu/ml/datasets/Drug+Review+Dataset+%28Druglib.com%29#

This Dataset has the reviews of different drugs from patients. Other features: side effect, ratings…

Python:

Hotel Review:

Kaggle directory:

https://www.kaggle.com/datafiniti/hotel-reviews?select=Datafiniti_Hotel_Reviews_Jun19.csv

This Dataset has the reviews of different hotels.

Other examples/applications: news, emails,

https://analyticsindiamag.com/10-popular-datasets-for-sentiment-analysis/

Amazon reviews: http://jmcauley.ucsd.edu/data/amazon/